ML PROJECT

Content-Based Recommendation System using Cosine Similarity

Submitted by: Tamana Farooq Khanday
Program: B.Tech (CSE)
Session: ML Project

Deployed Application: https://movie-recommendation-bqfvhbuotejgna7shxbmx3.streamlit.app/

GitHub Repository:https://github.com/tamannafarooq/Movie-Recommendation

---

Certificate Page (Template)

This is to certify that the project report titled **"Content-Based Recommendation System using Cosine Similarity"** is the bonafide work carried out by **Tamana Farooq** during the ML project period under the guidance of the project mentor and departmental supervision.

Supervisor Signature: _____
HOD Signature: _____
Date: _____

Self Declaration

I hereby declare that the work presented in this report titled **"Content-Based Recommendation System using Cosine Similarity"** is my original work carried out during ML project training. This report has not been submitted elsewhere for any other academic award.

I further declare that:
- All sources and references have been properly acknowledged.
- No part of this work has been plagiarized from any existing material.
- All code implementations, experiments, and results are genuine and reproducible.
- Any external libraries, frameworks, or datasets used are properly credited.

Name: Tamana Farooq Khanday

Signature: _____

Date: _____

Acknowledgement

I would like to express my sincere gratitude to my faculty mentors and internship guides for their valuable support during this ML project. Their guidance in machine learning concepts, project structuring, and deployment practices helped me complete this work successfully.

I also thank my peers and family for constant encouragement and feedback during development and testing.

Additionally, I acknowledge the open-source communities behind Python, Scikit-learn, Streamlit, and Pandas for providing robust tools that enabled this implementation. Special thanks to the TMDB API team for making movie metadata accessible for this demonstration project.

Finally, I appreciate the departmental resources and computing infrastructure that supported the development and deployment of this application.

Abstract

Recommendation systems are widely used in digital platforms to personalize user experience and improve content discovery. This project presents a **Content-Based Movie Recommendation System** that suggests similar movies based on textual attributes such as genres, keywords, and overview.

The system preprocesses movie metadata, converts text into vector representations using **CountVectorizer** or **TF-IDF**, and computes pairwise similarity using **Cosine Similarity**. Based on the selected movie, the application returns top-N relevant recommendations.

A complete interactive web application was developed using **Streamlit**, including recommendation cards, optional poster integration using TMDB API, and an evaluation dashboard with proxy metrics (Genre Match@K, Catalog Coverage, Mean Similarity). The project is deployed online and accessible for real-time demonstration.

Executive Summary

This project demonstrates the practical application of machine learning concepts to build a real-world recommendation system. The system successfully bridges the gap between theoretical ML knowledge and production-grade software engineering by incorporating best practices such as caching, API integration, and comprehensive evaluation metrics.

The content-based approach using cosine similarity on textual features provides interpretable recommendations without requiring user interaction data. The interactive Streamlit interface makes the system accessible to non-technical stakeholders while the modular codebase enables future enhancements and research extensions.

Key achievements include:
- End-to-end pipeline from data loading to deployment
- Interactive web application with multiple analysis views
- Quality evaluation framework for model assessment
- Live deployment with optional real-time poster integration
- Comprehensive documentation for reproducibility

Technical Highlights

5.4 Code Quality and Best Practices

- **Data Validation**: CSV columns are validated before processing; missing

columns raise informative errors.
- **Text Preprocessing**: Multi-stage cleaning pipeline removes special characters, normalizes case, and standardizes whitespace.
- **Resource Caching**: Streamlit cache decorators (`@st.cache_data`, `@st.cache_resource`) minimize recomputation and improve UX responsiveness.
- **Error Handling**: API calls include timeout protection and graceful fallbacks when external services are unavailable.
- **Modular Design**: Separate functions for data loading, feature engineering, model building, and evaluation enable testing and reusability.

5.5 Performance Considerations

- Similarity matrix computed once and cached locally; subsequent runs load from pickle artifact.
- API requests for poster URLs cached with 1-hour TTL to reduce TMDB API quota usage.
- Vectorizer max_features parameter tunable via sidebar slider to balance memory and granularity.
- Session state management prevents redundant user information collection.

6.4 Comparative Analysis

Metric	TF-IDF	CountVectorizer
Common Word Weight	Lower	Equal
Rare Term Emphasis	Higher	None
Sparsity	Higher	Lower
Recommendation Clarity	Better	Less distinctive
Computation Cost	Slightly higher	Lower

7.5 Integration with Version Control

- Code hosted on GitHub with complete commit history.
- README.md provides setup instructions and project description.
- requirements.txt lists all dependencies with pinned versions for reproducibility.
- .gitignore excludes large artifacts and API keys.

7.6 CI/CD and Automation

- Streamlit deployment directly from GitHub (automatic redeploy on push).
- Environment variables configured securely via Streamlit secrets management.
- No manual build steps required for deployment.

Table of Contents

9. References
10. Appendix

---

1. Introduction

1.1 Background

Modern users are exposed to massive content catalogs. Finding relevant items manually is difficult and time-consuming. Recommendation systems solve this by automatically ranking items based on user context or item similarity.
The evolution of recommendation systems has transformed how users discover content across platforms like Netflix, Amazon, and Spotify. Traditional approaches relied on manual curation or simple popularity ranking, but modern systems leverage machine learning to provide personalized suggestions at scale. Content-based filtering emerged as one of the foundational techniques in this domain, offering interpretability and efficiency when detailed user interaction data is unavailable.

1.2 Need for the Project

In many academic mini-projects, recommendation systems are explained theoretically but not deployed as usable products. This project focuses on both machine learning logic and practical web deployment.
This project bridges that gap by delivering a fully functional, deployed application that incorporates industry best practices such as caching, error handling, modular code organization, and comprehensive documentation. By combining theoretical ML concepts with real-world engineering considerations, it serves as a practical portfolio piece demonstrating readiness for professional software development roles.

1.3 Domain

- Machine Learning
- Natural Language Processing (basic text feature engineering)
- Recommender Systems
- Web App Development

---

2. Problem Statement and Objectives

2.1 Problem Statement

Given a movie title, recommend similar movies based on their content features.

2.2 Objectives

- Build an end-to-end content-based recommender system.
- Use robust text preprocessing and vectorization.
- Implement cosine similarity ranking logic.
- Provide an interactive Streamlit interface.
- Add quality-oriented features such as evaluation metrics and artifact caching.
- Deploy the project for live demonstration.

2.3 Scope

- Focused on movie recommendation using metadata.
- Does not use user watch history (non-collaborative model).
- Works with custom CSV and can be extended to larger TMDB datasets.

---

3. Literature Survey

3.1 Overview of Recommendation Approaches

- Content-Based Filtering: recommends items similar to selected item attributes.
- Collaborative Filtering: uses behavior patterns of similar users.
- Hybrid Systems: combines both to improve accuracy and robustness.

3.2 Why Content-Based for This Project

- Easier to explain in viva and implement from scratch.
- Does not require user-item interaction matrix.
- Works well with textual metadata fields.

3.3 Related Concepts

- Text preprocessing (cleaning, stop words, token features)
- Vectorization (CountVectorizer, TF-IDF)
- Similarity functions (Cosine Similarity)

---

4. System Design and Methodology

4.1 Dataset and Features

Input columns used:
- title
- genres
- keywords
- overview

Feature engineering:
- Merge genres + keywords + overview into a single text field called `tags`
- Clean and normalize text (lowercasing, punctuation removal, whitespace normalization)

4.2 Vectorization

Two selectable methods in UI:
- CountVectorizer
- TF-IDF Vectorizer

4.3 Similarity Computation

Cosine similarity formula:

$$
\text{cosine}(A, B) = \frac{A \cdot B}{\|A\|\|B\|}
$$

Where A and B are text feature vectors of two movies.

4.4 Recommendation Logic

1. Find index of selected movie.
2. Fetch similarity scores against all movies.
3. Sort scores in descending order.
4. Return top-N similar titles excluding selected title.

4.5 Advanced Components Implemented

- Local model artifact caching (`artifacts/`) for faster repeated runs.
- Optional TMDB poster retrieval by API key.
- Multi-tab app with Recommend, Evaluate, Explore Data, Viva Notes.

---

5. Implementation Details

5.1 Technology Stack

- Python
- Pandas
- NumPy
- Scikit-learn
- Streamlit
- Requests

5.2 Project Structure

- app.py
- movies.csv
- requirements.txt
- README.md
- .streamlit/config.toml
- .streamlit/secrets.toml.example
- artifacts/ (auto-generated cache)

5.3 Key Modules

- Data loading and validation
- Feature preprocessing
- Model build/load with cache
- Recommendation generation
- Poster API integration
- Evaluation metrics display

---

6. Results and Evaluation

6.1 Functional Results

- User can select a movie from dropdown and get top recommendations.
- Similarity score is shown for each recommendation.
- Poster cards displayed when TMDB key is provided.

6.2 Evaluation Dashboard

Implemented proxy quality metrics:
- Genre Match@K
- Catalog Coverage
- Mean Similarity

These metrics help explain model behavior and quality in viva presentations.

6.3 Observations

- TF-IDF generally gives cleaner recommendations than raw counts on text-heavy fields.
- Increasing max_features can improve granularity but increases computation time.
- Caching significantly reduces repeated startup computation.

---

7. Deployment and User Manual

7.1 Live Deployment

App URL: https://movie-recommendation-bqfvhbuotejgna7shxbmx3.streamlit.app/

7.2 Steps to Run Locally

1. Install dependencies:
   - `pip install -r requirements.txt`
2. Start app:
   - `streamlit run app.py`
3. Open local URL shown in terminal.

7.3 Optional Poster Setup

1. Create TMDB API key.
2. Set environment variable `TMDB_API_KEY`.
3. Enable poster option in sidebar.

7.4 User Flow

1. Open app.
2. Choose vectorizer and settings.
3. Select movie.
4. Click Recommend.
5. View recommendations and quality metrics.

---

8. Conclusion and Future Scope

8.1 Conclusion

The project successfully demonstrates an end-to-end content-based recommender system with a production-style interface and live deployment. It combines ML fundamentals with practical software engineering and provides clear explainability for academic evaluation.

8.2 Future Scope

- Add collaborative filtering model and hybrid ranking.
- Integrate user feedback for personalization.
- Expand dataset to full TMDB metadata.
- Add A/B comparison dashboard for model variants.

---

9. References

1. Scikit-learn Documentation: https://scikit-learn.org/stable/
2. Streamlit Documentation: https://docs.streamlit.io/
3. TMDB API Docs: https://developer.themoviedb.org/docs
4. Recommender Systems Overview (research references and survey papers)

---

10. Appendix

Appendix A: Screenshots to Attach in Final Submission

- Home tab (Recommend)
- Evaluation dashboard
- Dataset explorer
- Deployed app page
- GitHub repository page

Appendix B: Viva Questions (Quick Answers)
1. Why cosine similarity?
   - Cosine similarity measures the angle between two vectors in high-dimensional space, making it ideal for text data. It produces values between -1 and 1 (or 0 and 1 for normalized vectors), where 1 indicates identical direction. Unlike Euclidean distance, cosine similarity is invariant to vector magnitude, meaning longer documents don't automatically appear more similar. For sparse text vectors (common in NLP), it efficiently ignores zero-valued dimensions, reducing computation. This makes it particularly effective for comparing movie metadata vectors where sparsity is prevalent.

2. What is vectorization?
   - Vectorization is the process of converting text data into numeric feature vectors that machine learning algorithms can process. Raw text is unstructured; ML models require numerical input. Vectorization involves tokenizing text into words or n-grams, then mapping each token to a numeric representation. CountVectorizer creates frequency-based vectors (word counts), while TF-IDF vectors emphasize informative terms by downweighting common words. The resulting vectors are typically high-dimensional and sparse, capturing semantic meaning through statistical patterns. This numeric representation enables similarity computations, clustering, classification, and other ML operations on textual data.

3. Difference between CountVectorizer and TF-IDF?
   - CountVectorizer represents each document as a vector of raw term frequencies (word counts). Every occurrence of a word contributes equally, regardless of its informativeness. TF-IDF (Term Frequency-Inverse Document Frequency) weights each term by its frequency in the document (TF) and inversely by its frequency across all documents (IDF). Rare, informative words receive higher weights, while common words (like "the", "a") receive lower weights. In movie recommendation, TF-IDF typically produces cleaner, more distinctive recommendations because it emphasizes unique genre-keyword combinations over generic overlap. CountVectorizer is simpler and faster but may include noisy common-word matches.

4. Why content-based instead of collaborative filtering?
   - Content-based filtering recommends items similar to a selected item based on item attributes alone, without requiring user interaction history. Collaborative filtering relies on patterns across many users' watch histories, requiring large datasets and creating cold-start problems for new items or

users. For this academic project, content-based filtering is advantageous because: (1) it works with limited movie metadata (genres, keywords, overview), (2) it requires no historical user data, (3) recommendations are directly explainable by comparing item features, (4) implementation is straightforward enough for a solo project, and (5) it avoids privacy concerns associated with user behavior tracking. This approach prioritizes educational clarity and practical deployability over maximum accuracy.

Resume Description (Ready to Use)

Developed and deployed a content-based movie recommendation system using Python, Scikit-learn, and Streamlit. Implemented text preprocessing, TF-IDF/Count vectorization, cosine similarity ranking, local artifact caching, and an evaluation dashboard for model quality analysis.